

기존설계 스텝 패턴 — 프로세스 · 알고리즘 · 데이터 생성 · 자동설계 활용

Routing3D · 출발(EQUIP)/종단(DUCT) 스텝 학습 기술 레퍼런스 · 단위 mm · 기본 셀 50mm

1. 개요 — 스텝(stub)이란

Routing3D 엔진은 PostgreSQL(AUTOROUTINGV7)의 BIM 장애물·메인장비 PoC 를 입력으로 충돌 없는 직교 배관 경로를 자동 산출한다. 한 배관 경로는 '양 끝의 짧은 진출/진입 구간'과 '가운데 자유공간 구간'으로 나눌 수 있는데, 이 양 끝 구간을 스텝(stub)이라 부른다.

- 출발 스텝(EQUIP) : 출발 PoC(메인 장비) 쪽. 장비에서 '어느 면으로 나와 어느 방향으로 꺾여 주배관랙/자유공간으로 올라타는가'의 국소 형상.
- 종단 스텝(DUCT) : 종단 PoC(덕트·레터럴) 쪽. 자유공간에서 '어느 축으로 접근해 어느 면(주로 상부 +z)으로 진입하는가'의 국소 형상.

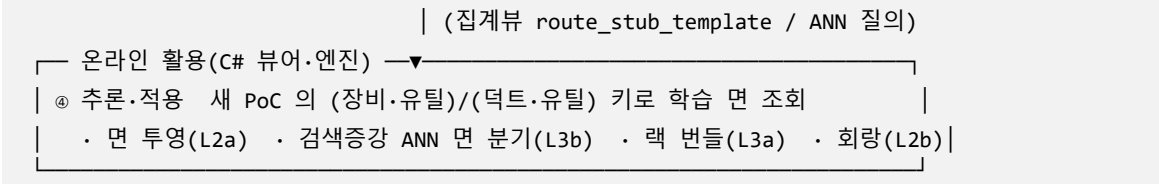
경로의 꺾임·관통·실패는 거의 전적으로 이 양 끝 혼잡부(스텝)에서 발생하고, 중간은 빈 공간 직선이 대부분이다. 따라서 '스텝만 사람 설계답게' 처리하면 경로 품질의 대부분이 결정된다. 본 문서는 사람이 그린 기존배관(TB_ROUTE_PATH)에서 스텝 형상을 학습해 자동설계에 활용하는 메커니즘을 기술한다.

단위는 모두 mm, 좌표는 BIM 과 동일한 월드 프레임이다. 본 문서의 모든 수치/규약은 구현 코드와 1:1 대응한다(코드 출처는 표지 주석 참조).

2. 전체 프로세스

학습은 오프라인(Python) 1 회, 활용은 라우팅 때마다(C# 뷰어/엔진) 일어난다. 4 단계 파이프라인이다.

```
— 오프라인 학습(Python) —
| ① 추출   TB_ROUTE_PATH 폴리라인 + 장비/덕트 AABB → 양 끝 스텝 잘라내기 |
| ② 정규화 앵커(장비/덕트) AABB 로컬 프레임으로 → 면·방향열·rise·offset |
| ③ 적재   특징벡터(24D)+방향(3D) → pgvector 저장소 route_stub_pattern |
```



단계	주체	산출물/효과	코드
① 추출	Python	배관당 출발·종단 스텝 점열	pattern_learn.learn_pipe
② 정규화	Python	face·dir_seq·rise·offset·feat(24D)·dir_unit(3D)	pattern_learn._make_sample
③ 적재	Python	route_stub_pattern 행 + 집계뷰 route_stub_template	pattern_db.insert_samples
④ 추론·적용	C#/엔진	PoC 면 투영·회랑·락·ANN	PatternStore · SceneViewModel

3. 입력 데이터 — 기존배관·앵커

학습 입력은 (a) 사람이 그린 기존 설계배관 폴리라인과 (b) 그 양 끝이 닿는 앵커(장비/덕트) AABB 다. C# ObstacleDbLoader 의 로직을 Python(route_db.py)으로 1:1 포팅해 동일 월드 프레임에서 읽는다.

테이블	역할	핵심 컬럼
TB_ROUTE_PATH (+_SEGMENTS/_SEGMENT_DETAIL)	기존 배관 폴리라인 + 양 끝 PoC	ROUTE_PATH_GUID, SOURCE/TARGET_POS, SOURCE_UTILITY, UTILITY_GROUP, SOURCE_SIZE
TB_BIM_EQUIPMENT	출발 앵커(메인 장비 AABB)	IS_MAIN, NAME, MIN/MAX_X/Y/Z
TB_DUCT_LATERAL	종단 앵커(덕트/레터럴 AABB)	CATEGORY, UTILITY, MIN/MAX_X/Y/Z

3-테이블 조인으로 폴리라인을 순서대로 잇고(ROUTE_PATH_GUID, s.ORDER, sd.ORDER), 같은 공정의 다른 tool 배관은 SOURCE_OWNER_POS(장비 위치)를 장애물 XY bbox 로 필터해 거른다. 각 배관은 출발 PoC 좌표(source_pos)·종단 PoC 좌표(target_pos)·유틸리티·그룹·호칭경(diameter_mm)을 갖는다.

앵커 매칭: 출발 PoC 는 그를 포함하는 장비(없으면 3000mm 내 최근접), 종단 PoC 는 그를 포함하는 덕트(없으면 3000mm 내 최근접)로 결정한다(find_equipment / find_duct, ANCHOR_MAX_MM=3000).

4. 스텝 추출 알고리즘

한 배관(learn_pipe)에서 최대 2 개 표본(출발 EQUIP·종단 DUCT)을 만든다. 절차는 다음과 같다.

1. 방향 정렬: 폴리라인을 출발 PoC 가 앞이 되도록 정렬(_oriented). 출발 인덱스 i_src·종단 인덱스 i_tgt 를 최근접점으로 찾고, i_tgt > i_src 가 아니면 표본을 만들지 않는다.
2. 출발 스텝: i_src→i_tgt 방향 점열로, 종단 스텝: i_tgt→i_src 역순 점열로 _walk_stub 을 돌린다.
3. 스텝 잘라내기(_walk_stub): PoC 에서 출발해 누적 길이가 STUB_MAX_MM(4000mm)을 넘거나 꺾임이 STUB_MAX_BENDS(3)에 도달하면 종료. 각 세그먼트를 직교 축으로 스냅(axis_snap)하고, 연속 동일 방향은 병합해 방향 시퀀스 dir_seq 를 만든다(꺾임 수 = len(dir_seq) - 1).
4. 면 분류(nearest_face): PoC 가 가장 가까운 앵커 AABB 면(0.5 = +x,-x,+y,-y,+z,-z)을 면(face)으로 한다.
5. 상승/오프셋: rise_mm = 면 법선축으로 스텝이 이동한 최대 거리(덕트 상부로 뜬 높이 등). offset_mm = PoC 와 면 평면 사이 간극(표면 바깥 여유).

축 스냅(axis_snap)은 3-벡터를 절대값이 가장 큰 축의 부호로 6 직교 축 인덱스(0..5)에 매핑한다. 이로써 약간 사선인 실측 폴리라인도 직교 방향열로 정규화된다.

```
axis_snap(d): ax = argmax_i |d[i]|; return ax*2 + (0 if d[ax]>=0 else 1)
# 예) (10, 5, 100) → 지배축 z(+) → 인덱스 4(+z)
```

스텝 형상(정규화 후 한 표본):

face	∈ {+x, -x, +y, -y, +z, -z}	PoC 가 닿는 앵커 면
dir_seq	예) "+z,+x"	진행 방향열(연속 병합), 꺾임 = 길이-1
rise_mm	예) 1200	면 법선축 최대 이동(랙 상승/덕트 진입 높이)
offset_mm	예) 50	PoC ~ 면 평면 간극
diameter_mm	SOURCE_SIZE	파싱 외경(40A=40, 1/2B=12.7 ...)

도메인 규칙 입증(project6 실측): 출발 스텝은 face=-z(장비 아래로 빠져나감)가, 종단 스텝은 face=+z(덕트 상부로 진입)가 지배적이다 — pytest test_db_learn_project6 가 이 분포를 자동 검증한다.

5. 특징벡터(24 차원) 설계

스텝을 (앵커종류·유틸그룹·유틸)이라는 '범주 키'와 24 차원 기하 특징벡터(feat)-3 차원 진행 단위벡터(dir_unit)로 표현한다. 범주는 절대 제약(유틸 위반 금지)이라 질의 시 WHERE 로 먼저 거르고, 그 안에서만 벡터 검색으로 형상이 얇은 스텝을 고른다.

구간	차원	내용
face 1-hot	6	PoC 가 닿는 앵커 면(+x..-z)
1 차 방향 1-hot	6	dir_seq[0] (스텝 첫 진행 측)
2 차 방향 1-hot	6	dir_seq[1] (없으면 전부 0)
앵커내 상대좌표	3	(poc - anchor_min)/(anchor_max - anchor_min), 축별 [0,1]
시작→종단 단위	3	unit(target - source). 종단 스텝은 부호 반전(앵커로의 접근방향)

one-hot·상대좌표·단위벡터라 성분 스케일이 균형을 이뤄, 그룹 내 L2(유클리드)/코사인 검색이 의미를 가진다. 차원은 pattern_db.FEAT_DIM(=24)·스키마 vector(24)·C# 6 차원 ANN 서브셋과 반드시 일치한다.

```
feat(24) = [face(6)] ++ [dir1(6)] ++ [dir2(6)] ++ [rel_pos(3)] ++ [dir_unit(3)]
dir_unit(3) = unit(target - source)          # EQUIP 표본
              = -unit(target - source)       # DUCT 표본(앵커로의 접근방향)
검색증강(C#) 서브셋 = [rel_pos(3), dir_unit(3)] # 면 분기에 쓰는 6 차원
```

6. 패턴 데이터 생성 — pgvector 저장소

추출·정규화된 표본은 PostgreSQL + pgvector 저장소에 영속화된다. 스키마 단일 출처는 db/schema/route_stub_pattern.sql 이고, pattern_db.apply_schema() 가 그 파일을 실행한다(먹등).

6.1 테이블 route_stub_pattern

컬럼군	컬럼	설명
출처	source_file, route_path_guid, anchor_kind, anchor_name	anchor_kind ∈ {EQUIP, DUCT}
범주 키	utility_group, utility	질의 pre-filter(절대 제약)
원자료 좌표	poc_pos, anchor_min, anchor_max	double precision[3] (mm)

정규화 형상	face, dir_seq, n_bends, rise_mm, offset_mm, diameter_mm	스텝 형상
벡터	dir_unit vector(3), feat vector(24)	pgvector 컬럼

인덱스: ix_rsp_key(btree, anchor_kind/utility_group/utility 범주 pre-filter 용),
 ix_rsp_feat_hnsw(HNSW, vector_l2_ops — feat 최근접), ix_rsp_dir_hnsw(HNSW,
 vector_cosine_ops — 방향 최근접). pgvector-cube-postgis 확장 설치 전제.

6.2 적재 규약

- psycopg2 는 vector 타입을 모르므로, 벡터를 '[a,b,c]' 텍스트 리터럴(vec_literal)로 바인딩하고 ::vector 로 캐스트한다. double[] 컬럼은 파이썬 리스트로 바인딩.
- 역등 재학습: 같은 프로젝트를 다시 학습할 때 clear_source(source_file)로 기존 표본을 지운 뒤 insert_samples 로 새로 넣는다(중복 누적 방지).

6.3 집계뷰 route_stub_template

키(anchor_kind, utility_group, utility)마다 대표값을 미리 집계한 뷰다 — 최빈 면(mode), 대표 rise/offset(percentile), 평균 꺾임, 표본수 n. C# 뷰어가 이 뷰를 읽어 '키별 1 개 대표 스텝'을 빠르게 조회한다(L2a 의 1 차 추론 경로).

6.4 랙 레벨 학습(L3a 보조 데이터)

스텝과 별개로, 기존배관 '수평 런'이 모이는 z-높이(파이프 랙)를 유틸그룹별로 학습한다(learn_rack_levels). 각 폴리라인 세그먼트 중 수평($|dz| \leq 0.34 \times \text{수평거리}$)이고 800mm 이상인 것만 채택해 중점 z 를 100mm 버킷으로 양자화하고, 그룹별 누적 런 길이를 집계한다. 실측 project6: Gas 83%@z=12200 · Exhaust 97%@z=13400 · Waste Liquid 71%@z=14800 등 강한 랙 집중이 드러난다.

7. 패턴을 자동설계에 활용하는 방법

추론은 새 라우팅 작업의 PoC 에 학습 결과를 적용하는 단계다. 엔진(C++)은 변경하지 않고, 뷰어(C#)가 PoC 투영 방향·회랑 셀·랙 레벨을 결정해 엔진에 넘긴다. 네 가지 활용 메커니즘이 독립 토글로 조합된다.

7.1 학습면 PoC 투영 (L2a)

PatternStore.TryGet(anchor_kind, group, utility)로 집계뷰의 대표 면을 조회하고, 그 면으로 PoC 를 솔리드 표면 바로 바깥(+½셀)으로 투영한다(LiftPocToSurface(preferFace)). 즉 '최근접 면'이라는 고정 규칙 대신 '학습된 면'(장비=-z 하부, 덕트=+z 상부 등)으로 빼낸다. 키 미스/저장소 부재 시 자동으로 기존 최근접-면 규칙으로 폴백(무해).

7.2 검색증강 ANN 면 분기 (L3b)

한 (그룹·유틸) 키 안에 진입면이 여럿 섞인 '다중면(bimodal) 키'(예 UPW_S = +x 57% / +z 43%)에서는 집계 대표면이 일부 PoC 에 틀린다. 이때 그 PoC 와 가장 닮은 학습 표본을 [상대좌표(3), 접근방향(3)] 6 차원 최근접으로 찾아 그 면을 쓴다(PatternStore.TryGetFaceAnn → SceneViewModel.LearnedDuctFace).

단, 무분별한 ANN 은 오히려 해롭다(실측: project6 199→192) — 일부 키는 [rel,dir]이 면과 무상관/반상관이기 때문이다. 그래서 적재 시 키마다 leave-one-out 으로 'ANN vs 집계 다수결' 정확도를 비교해, ANN 이 +10pp 이상 이기는 키에서만 ANN 을 적용하는 자기검증 게이트를 둔다. 실측: NFW 만 통과(ANN 97% vs 집계 59%), UPW_S/HOT DI_S 탈락 → 집계 유지.

7.3 유틸그룹 랙 번들링 (L3a)

6.4 에서 학습한 그룹 랙 z-높이를 엔진 rack_levels(= w_corridor 면제 z-셀)로 넘긴다. 엔진은 랙 z-셀 밖을 셀당 가산(부드러운 w_corridor)하므로, 같은 그룹의 새 배관이 공용 랙 높이에 뭉친다. 랙 페널티는 회랑(0.5)보다 부드러운 cell×0.2(0.5 는 한 배관을 막아 성공이 떨어짐). 실측: 199→200, 랙 z-집중도 17→21%, 총길이 감소.

7.4 기존설계 회랑 소프트바이어스 (L2b)

매칭되는 기존배관(FindMatchingExistingPipe — 양 끝 PoC 거리합 최소) 폴리라인을 격자 셀로 복셀화해 엔진 회랑 셀로 주입한다(r3d_set_corridor_cells). 엔진은 회랑 밖을 w_corridor 로 가산하므로 새 경로가 사람 설계를 부드럽게 따라간다(충돌은 여전히 회피). 옵트인(기본 OFF) — 경로 모양을 바꾸기 때문.

7.5 엔진 비용함수 연계 (cost.hpp)

회랑·랙은 모두 엔진 CostModel 의 같은 메커니즘을 쓴다: cell_penalty 가 'rack 레벨 U corridor 셀'에 속하지 않는 셀에 w_corridor 를 가산한다. 보너스가 아닌 '회랑/랙 밖 가산'이라 admissibility(휴리스틱 하한성)가 보존된다. 회랑 키는 64 비트(occ.lin)라 정밀·거대 격자에서도 무손실이다. w_corridor=0 이면 전혀 동작하지 않아 골든(기준) 결과가 불변이다.

8. 실측 결과 (project6, cell=100, 208 작업)

동일 실행에서 각 활용 메커니즘을 누적 적용한 헤드리스 A/B 측정(--dbroute). weighted A* w_heur=2.0 기준.

단계	성공/208	비고
기하 baseline(패턴 OFF)	187	고정 규칙만
+ 학습면 투영(L2a)	187	면 정정·헛탐색 제거(38s→23s)
+ 접근불가 PoC 스냅(전처리)	199	파문힌 PoC → 최근접 자유셀
+ 랙 번들(L3a)	200	랙 z-집중도 17→21%, 총길이 ↓
+ 회랑(L2b, 옵트인)	199(동일)	설계추종(총길이 ↓), 모양 변화
+ 검색증강 ANN(L3b)	199(유지)	NFW 진입면 97% 정확, 회귀 0

성능: 회랑 ON 의 탐색 폭증(이전 47s)은 weighted A* w_heur 1.5→2.0 으로 6.2s 로 단축(약 7.6×) — 회랑 비용장을 휴리스틱에 반영해 탐색을 목표지향으로 만든 결과다. 남은 8 실패는 혼잡/막힘(경로 없음)으로 패턴 범위 밖이며 rip-up/CBS 영역이다.

9. 운영 — CLI·검증

```
# 학습 + 검수 리포트(콘솔, DB 미적재)
python -m routing3d_py.pattern_learn --project 6 --report
# 유틸그룹 랙 레벨(수평 런 z-높이) 리포트(L3a)
python -m routing3d_py.pattern_learn --project 6 --rack-report
# 학습 + pgvector 저장소 적재(기존 표본 정리 후, --apply-schema 자동)
python -m routing3d_py.pattern_learn --project 6 --write-db
# 스키마 적용 / 저장소 통계(키별 대표 템플릿)
python -m routing3d_py.pattern_db --apply-schema
python -m routing3d_py.pattern_db --stats
# 헤드리스 A/B(각 메커니즘 on/off): env R3D_PATTERNS/SNAP/CORRIDOR/RACK/ANN
Routing3D.Viewer.exe --dbroute 6 100 ALL out.txt
```

- 추론 프리미티브(검색증강): pattern_db.suggest_stub(kind, group, util, feat, k) → StubSuggestion(거리가중 K-NN 투표 면 + 평균 rise/offset + 신뢰도).

- 테스트: pytest 순수 로직 9 건(axis_snap·nearest_face·feat 차원·walk_stub·is_horizontal·learn_rack_levels ...) + DB 통합 3 건(도메인 분포·ANN 범주필터·suggest 합의).
- 뷰어 토글: '기존설계 패턴'(L2a, 기본 ON) · '기존설계 회랑(L2b)'(기본 OFF) · '랙 번들(L3a)'(기본 OFF). 검색증강(L3b)은 다중면 키에서 자동(게이트 통과 키만).

10. 한계 · 향후

- 스텝 분기 특징의 한계: 한 키 내 진입면 분기가 [rel,dir]로 항상 예측되진 않는다(UPW_S/HOT DI_S 반상관). 접근 세그먼트 방향열·인접 장애물 등 특징 보강으로 분기 정밀도를 높이는 것이 다음 과제.
- 검색증강 본격화(L3): 현재 ANN 은 면 분기에 한정. rise/offset/dir_seq 까지 컨텍스트 임베딩으로 검색해 스텝 형상 전체를 재현하는 검색증강 라우팅(pgvector HNSW 본격 활용)으로 확장 가능.
- 혼잡 잔여 실패: 양 끝 스텝이 아닌 중간 혼잡으로 인한 실패(경로 없음)는 패턴 범위 밖 — negotiated-congestion/CBS(비용기반 충돌회피)가 후속.
- 신규 DB(DDW_AI_DB) 전환 시 로더(route_db/obstacle_db) 재작성 필요(스키마 전면 상이).